

یادداشت آموزشی پروژه IEEE-CIS Fraud Detection

خلاصه روند پروژه، feature engineering، مدل سازی، ensemble، و تصمیم نهایی

نتیجه نهایی پروژه: مدل نهایی از نظر معیار اصلی مسابقه، یعنی ROC-AUC، یک ensemble در نسخه v7 است.

Final Champion: v7 Final Ensemble = 0.70 LightGBM + 0.20 XGBoost + 0.10 CatBoost

بهترین ROC-AUC validation: 0.9292

۱. مسئله چه بود؟

مسئله تشخیص تقلب در تراکنش ها بود. دیتاست مربوط به مسابقه IEEE-CIS Fraud Detection در Kaggle است. هر ردیف یک تراکنش است و هدف این است که تشخیص دهیم آن تراکنش fraud هست یا نه.

معنی	ستون هدف
تراکنش عادی	isFraud = 0
تراکنش تقلبی	isFraud = 1

داده شدیداً نامتوازن بود. حدود ۳.۵٪ تراکنش ها fraud بودند. بنابراین accuracy به تنهایی معیار خوبی نبود. معیار اصلی مسابقه ROC-AUC است، اما ما علاوه بر آن F1، LogLoss، PR-AUC و threshold analysis را هم بررسی کردیم.

۲. فایل های خام و merge اصلی

فایل	نقش
train_transaction.csv	جدول اصلی تراکنش های train؛ شامل ستون هدف isFraud
train_identity.csv	اطلاعات identity/device برای بخشی از تراکنش های train
test_transaction.csv	جدول اصلی test؛ بدون ستون هدف
test_identity.csv	اطلاعات identity/device برای بخشی از تراکنش های test

ما transaction و identity را با ستون TransactionID merge کردیم. چون identity فقط برای بخشی از تراکنش ها وجود داشت، یک feature به نام has_identity ساختیم تا مدل بداند کدام تراکنش اطلاعات identity دارد.

۳. validation چرا time-aware بود؟

ستون TransactionDT زمان نسبی تراکنش را نشان می دهد. در fraud detection، آینده نباید وارد train شود؛ وگرنه leakage رخ می دهد. به همین دلیل داده ها را بر اساس زمان مرتب کردیم و split را این طور انجام دادیم:

بخش	روش
Train	۸۰٪ اول زمانی
Validation	۲۰٪ آخر زمانی

این روش سخت تر از random split است، اما واقعی تر و قابل دفاع تر است.

۴. مسیر نسخه‌ها از v1 تا v7

نسخه	هدف	اضافه شده‌ها
v1	ساخت baseline تمیز	ستون‌های اصلی، C/D/M محدود، card، amount، time و identity محدود
v2	تست ارزش email features	same_email_domain و missing flags، R_email_group، P_email_group
v3	ساخت feature از مبلغ تراکنش	log، اعشار، round، cents، بودن مبلغ
v4	استفاده از frequency کارت	card1_count
v5	گسترش جدی featureها	تقریباً همه transaction و identity features، به صورت memory-safe و بدون one-hot سنگین
v6	افزودن aggregation و interaction count	count ترکیبی و آمار مبلغ بر اساس card، addr، email و product
v7	نسخه نهایی	aggregation، UID-style features، قوی‌تر، CatBoost، XGBoost، LightGBM و weighted ensemble

۵. featureهایی که خودمان ساختیم

۵.۱. ویژگی‌های مربوط به زمان

توضیح	Feature
شماره روز تراکنش از روی TransactionDT	TransactionDay
ساعت تراکنش، از ۰ تا ۲۳	TransactionHour
هفته نسبی تراکنش	TransactionWeek

این featureها به مدل کمک کردند تغییرات زمانی، drift، و الگوی تقلب در ساعت‌ها یا دوره‌های خاص را ببیند.

۵.۲. ویژگی‌های مربوط به device و identity

توضیح	Feature
آیا ردیف مربوطه در فایل identity وجود داشته یا نه	has_identity
گروه‌بندی سیستم عامل از id_30، مثل windows، android، ios	OS_group
گروه‌بندی مرورگر از id_31، مثل chrome، safari، firefox	Browser_group
گروه‌بندی device، مثل samsung، huawei، apple_mobile	DeviceInfo_group
عرض صفحه نمایش استخراج‌شده از id_33	screen_width
ارتفاع صفحه نمایش استخراج‌شده از id_33	screen_height

ما raw ستون‌های خیلی شلوغ مثل DeviceInfo، id_30، id_31 و id_33 را مستقیم وارد مدل نکردیم. آن‌ها را به گروه‌های ساده‌تر و قابل فهم‌تر تبدیل کردیم.

۵.۳. ویژگی‌های مربوط به email

توضیح	Feature
آیا ایمیل خریدار missing است؟	P_email_missing
آیا ایمیل گیرنده missing است؟	R_email_missing
آیا دامنه ایمیل خریدار و گیرنده یکی است؟	same_email_domain
گروه‌بندی دامنه ایمیل خریدار، مثل gmail، yahoo، hotmail، anonymous	P_email_group
گروه‌بندی دامنه ایمیل گیرنده	R_email_group

در EDA دیدیم وجود یا نبود R_emaildomain و همچنین برابر بودن دامنه‌های email می‌تواند سیگنال مهمی برای fraud باشد. در نسخه‌های بعدی، خود P_emaildomain و R_emaildomain هم با ordinal encoding وارد مدل شدند.

۵.۴. ویژگی‌های مربوط به مبلغ تراکنش

توضیح	Feature
لگاریتم مبلغ تراکنش با log1p؛ برای کاهش skew	TransactionAmt_log
بخش اعشاری مبلغ؛ مثلاً 68.5 می‌شود 0.5	TransactionAmt_decimal
آیا مبلغ عدد رند است؟	TransactionAmt_is_round
بخش cents یا اعشار مبلغ، بین 0 تا 99	TransactionAmt_cents

در feature importance مشخص شد TransactionAmt_decimal و TransactionAmt_log واقعاً برای مدل مفید بودند.

۶. Frequency features چه بودند؟

frequency feature یعنی تعداد دفعات دیده شدن یک مقدار در train. مثلاً اگر مقدار خاصی از card1 در train ده هزار بار دیده شده باشد، مقدار card1_count برای آن ردیف ده هزار می شود.

معنی	Feature
تعداد تکرار مقدار card1 در train	card1_count
تعداد تکرار مقدار card2	card2_count
تعداد تکرار مقدار card3	card3_count
تعداد تکرار مقدار card5	card5_count
تعداد تکرار مقدار addr1	addr1_count
تعداد تکرار مقدار addr2	addr2_count

این feature ها leakage-safe ساخته شدند؛ یعنی شمارش فقط روی train انجام شد و سپس روی validation map شد. اگر مقدار جدیدی در validation وجود داشت که در train نبود، count آن صفر شد.

۷. UID یعنی چه و دقیقاً چطور ساخته شد؟

UID یعنی یک شناسه ترکیبی. در این پروژه، هدف UID این بود که یک تراکنش را فقط با یک ستون مثل card شناسیم، بلکه ترکیب کارت، آدرس، ایمیل و محصول را همزمان ببینیم. این کار در fraud detection مهم است، چون رفتار غیرعادی معمولاً در ترکیب چند نشانه ظاهر می شود، نه در یک ستون تنها.

برای ساخت UID، هر مقدار ابتدا به متن تبدیل شد. اگر مقدار missing بود، به کلمه Missing تبدیل شد. بعد مقادیر با علامت underscore به هم وصل شدند.

معنی ساده	روش ساخت دقیق	UID feature
ترکیب کارت و آدرس	<code>str(card1) + "_" + str(addr1)</code>	uid_card1_addr1
ترکیب دو مشخصه کارت و آدرس	<code>str(card1) + "_" + str(card2) + "_" + str(addr1)</code>	uid_card1_card2_addr1
ترکیب کارت، آدرس و ایمیل خریدار	<code>str(card1) + "_" + str(addr1) + "_" + str(P_emaildomain)</code>	uid_card1_addr1_pemail
نسخه قوی تر با کارت، آدرس و ایمیل	<code>str(card1) + "_" + str(card2) + "_" + str(addr1) + "_" + str(P_emaildomain)</code>	uid_card1_card2_addr1_pemail
ترکیب کارت، آدرس و نوع محصول	<code>str(card1) + "_" + str(addr1) + "_" + str(ProductCD)</code>	uid_card1_addr1_product
نسخه قوی تر با کارت، آدرس و محصول	<code>str(card1) + "_" + str(card2) + "_" + str(addr1) + "_" + str(ProductCD)</code>	uid_card1_card2_addr1_product

مثلاً اگر `card1 = 12345`، `addr1 = 330` و `P_emaildomain = gmail.com` باشد، مقدار `uid_card1_addr1_pemail` تقریباً به این شکل ساخته می شود:

12345_330_gmail.com

بعد برای همین UIDها، frequency و aggregation ساختیم. این باعث شد مدل بفهمد یک ترکیب خاص کارت-آدرس-ایمیل چقدر رایج است و مبلغ تراکنش نسبت به رفتار معمول همان ترکیب چقدر غیرعادی است.

۸. Interaction count features چه بودند؟

interaction count شبیه frequency feature است، ولی روی ترکیب دو ستون ساخته می‌شود. مثلاً `card1_addr1_count` تعداد دفعات دیده‌شدن ترکیب `card1 + addr1` در `train` است.

معنی	Feature نمونه
تعداد دفعات ترکیب کارت و آدرس	<code>card1_addr1_count</code>
تعداد دفعات ترکیب کارت و نوع محصول	<code>card1_ProductCD_count</code>
تعداد دفعات ترکیب کارت و ایمیل خریدار	<code>card1_P_emaildomain_count</code>
تعداد دفعات ترکیب <code>card2</code> و آدرس	<code>card2_addr1_count</code>
تعداد دفعات ترکیب دامنه ایمیل و نوع محصول	<code>P_emaildomain_ProductCD_count</code>

در `v6` و `v7`، چندین `interaction count` جزو مهم‌ترین feature ها شدند. این نشان داد که ترکیب ستون‌ها برای کشف `fraud` مهم‌تر از نگاه تک‌ستونی است.

۹. Aggregation features چه بودند؟

aggregation یعنی رفتار معمول مبلغ تراکنش را برای یک گروه محاسبه کنیم. مثلاً برای هر مقدار `card1` بررسی کنیم میانگین، انحراف معیار، میانه، حداقل و حداکثر مبلغ تراکنش‌های آن کارت چقدر بوده است.

معنی	Feature
میانگین مبلغ تراکنش برای هر <code>card1</code>	<code>card1_TransactionAmt_mean</code>
انحراف معیار مبلغ برای هر <code>card1</code>	<code>card1_TransactionAmt_std</code>
میانه مبلغ برای هر <code>card1</code>	<code>card1_TransactionAmt_median</code>
کمترین مبلغ دیده‌شده برای هر <code>card1</code>	<code>card1_TransactionAmt_min</code>
بیشترین مبلغ دیده‌شده برای هر <code>card1</code>	<code>card1_TransactionAmt_max</code>

بعد از ساخت آمارهای گروهی، چند feature نسبی هم ساختیم:

معنی	Feature
مبلغ فعلی منهای میانگین مبلغ همان کارت	<code>TransactionAmt_minus_card1_mean</code>
نسبت مبلغ فعلی به میانگین همان کارت	<code>TransactionAmt_ratio_card1_mean</code>
غیرعادی بودن مبلغ نسبت به میانگین و انحراف معیار همان کارت	<code>TransactionAmt_zscore_card1</code>
جایگاه مبلغ در بازه حداقل تا حداکثر همان گروه	<code>TransactionAmt_range_ratio_card1</code>

همین منطق را برای گروه‌هایی مثل `card2`، `addr1`، `P_emaildomain`، `ProductCD` و `UID` ها هم اجرا کردیم.

نکته leakage: تمام `count` و `aggregation` ها فقط از `train` محاسبه شدند و بعد روی `validation map` شدند. این یعنی `validation` هیچ نقشی در محاسبه میانگین‌ها و `count` ها نداشت.

۱۰. Encoding و memory-safe pipeline

در نسخه‌های سنگین، one-hot encoding باعث مصرف RAM زیاد می‌شد. برای همین در v5 به بعد، ordinal ها را با categorical encoding تبدیل کردیم. mapping فقط از train ساخته شد و category های جدید در validation مقدار 1- گرفتند. ستون‌های عددی نیز با median مربوط به train پر شدند. اگر ستونی در train کاملاً missing بود، مقدار median آن با 999- جایگزین شد. سپس dtype ها به float32 و int32 کاهش یافتند تا مدل روی لپ‌تاپ قابل اجرا بماند.

۱۱. نتایج نسخه‌های اصلی

نسخه / مدل	ROC-AUC	PR-AUC	LogLoss	F1@0.50	Best F1
LightGBM v4	0.9082	0.4891	0.2782	0.3271	0.4805
LightGBM v5	0.9186	0.5436	0.2423	0.3770	0.5172
LightGBM v6	0.9198	0.5725	0.1932	0.4358	0.5568
LightGBM v7	0.9252	0.5977	0.1321	0.5223	0.5916
v7 Ensemble	0.9292	0.5905	0.1621	0.4981	0.5804

از نظر معیار اصلی مسابقه، یعنی ROC-AUC، بهترین مدل ما ensemble نسخه v7 بود. از نظر بهترین F1 عملیاتی، LightGBM v7 با threshold حدود 0.70 بهتر بود.

۱۲. مدل‌های v7 و ensemble

مدل	ROC-AUC	PR-AUC	LogLoss	F1@0.50
LightGBM v7	0.9252	0.5977	0.1321	0.5223
XGBoost v7	0.9223	0.5495	0.2310	0.4021
CatBoost v7	0.9112	0.5091	0.3049	0.3269

بهترین ensemble با جست‌وجوی وزن‌ها ساخته شد:

مدل	وزن
LightGBM	0.70
XGBoost	0.20
CatBoost	0.10

این ensemble بهترین ROC-AUC = 0.9292 را گرفت. اما LightGBM v7 از نظر PR-AUC، LogLoss و بهترین F1 عملیاتی قوی‌تر بود.

۱۳. Feature importance نسخه v7 چه گفت؟

در feature v7، top‌های نشان دادند که مدل واقعاً از UID و aggregation استفاده کرده است. چند feature مهم:

Feature	اهمیت	تفسیر
TransactionDT	1778	زمان تراکنش بسیار مهم بود
card1	1519	کارت/شناسه کارت هنوز بسیار مهم است
C13	1315	یکی از C features قوی بود
uid_card1_addr1_pemail_TransactionAmt_std	1047	پراکندگی مبلغ برای UID کارت-آدرس-ایمیل مهم بود
card2_addr1_count	1046	تکرار ترکیب card2 و آدرس قوی بود
uid_card1_addr1_pemail_TransactionAmt_max	1020	حداکثر مبلغ تاریخی آن UID مهم بود

این یعنی بهبود v7 تصادفی نبود. مدل از featureهای مهندسی‌شده‌ای استفاده کرد که دقیقاً برای شناسایی رفتار غیرعادی ساخته شده بودند.

۱۴. تصمیم نهایی پروژه

v7 Final Ensemble :**Final champion by ROC-AUC**

0.9292 :**ROC-AUC**

LightGBM 0.70 + XGBoost 0.20 + CatBoost 0.10 :**Weights**

LightGBM v7 :**Best single model**

0.1321 :**LogLoss**, 0.5977 :**PR-AUC**, 0.9252 :**ROC-AUC**

LightGBM v7 at threshold 0.70, F1 = 0.5916 :**Best operational F1**

بنابراین برای README باید دو نکته را جداگانه گزارش کنیم: برای metric مسابقه، ensemble نهایی بهترین است؛ برای تفسیر و کاربرد عملیاتی، LightGBM v7 مدل تکی اصلی است.

۱۵. مرحله بعدی پروژه

دیگر feature-engineering جدید انجام نمی‌دهیم. از اینجا پروژه باید جمع‌بندی و برای GitHub آماده شود:

- ساخت جدول نهایی مقایسه مدل‌ها و نسخه‌ها
- تمیزکاری پوشه‌ها و انتقال فایل‌های اضافی به archive یا experiments
- نوشتن README نهایی با توضیح feature engineering, leakage prevention, validation, و نتایج
- در صورت نیاز، ساخت فایل submission برای test
- commit نهایی در GitHub

این یادداشت برای مرور شخصی پروژه ساخته شده است. هدف آن این است که بعداً با خواندن آن دقیقاً یادت بیاید چه feature‌هایی ساختیم، چرا ساختیم، و نتیجه هر مرحله چه شد.